

CodeIntel

Local AI Code Intelligence — Quick Reference

What it is · What it uses · What each part does

What Is It?

A **web app** that runs locally, points a local AI model at your codebase, and produces structured findings — without sending your code anywhere.

Three outputs:

Mode	Output
Analysis	Bug / dead-code / rule findings with confidence ratings
Trace	Call-graph diagram + per-node summaries
Metrics	Complexity table — cyclomatic complexity, nesting, length

The handoff: findings save as a Markdown report into your repo. You paste a `#file:` reference into GitHub Copilot Chat to get tickets, fix plans, or PR descriptions.

Tech Stack

Runtime & Framework

.NET 10 (ASP.NET Core)

The server. Hosts both the API and the React app from a single process — one URL, one container.

React 19 + Vite

The browser UI. Vite serves it in dev; in production the .NET server serves it directly.

SignalR (WebSocket)

Pushes every token the AI generates to the browser in real time — no polling.

SQLite (WAL mode)

Stores analysis history, trace results, the findings cache, and the ignored-findings list. One file on disk, zero ops.

AI & Code Analysis

LLamaSharp

Runs a local AI model (`.gguf` file) in-process inside the .NET server. No Ollama, no external API — the model is part of the app.

GGUF model — Qwen 2.5 Coder 7B Q4

The AI model. 7 billion parameters, 4-bit quantized, ~4.7 GB on disk. Runs on CPU; partial GPU offload available via Vulkan or CUDA.

Roslyn (Microsoft.CodeAnalysis)

Microsoft's C# compiler API. Used to load `.sln` files, navigate symbols, find callers/callees, and extract method bodies — without running the code.

ANTLR4

Parser generator. Used to build a custom grammar for Oracle PL/SQL — extracts object references, table names, cursor declarations from stored-proc files.

Infrastructure

MUI (Material UI v9)

React component library. All UI elements — tables, chips, cards, icons — come from MUI.

Zustand

Lightweight React state store. Holds workspace state, analysis state, and trace state without Redux boilerplate.

TanStack Query

Manages HTTP requests from the browser — caching, loading states, refetch on focus.

Mermaid

JavaScript library that renders flowchart diagrams in the browser. The call-graph diagrams are generated as Mermaid syntax on the server and rendered client-side.

Language Backends

TypeScript LSP (`typescript-language-server`)

An external process the server talks to over standard JSON-RPC. Gives Roslyn-equivalent features for TypeScript files — symbol lookup, call hierarchy, go-to-definition.

Language Backend Registry (`ILanguageBackendRegistry`)

The interface that all language-specific features (C#, TypeScript, PL/SQL) implement. The rest of the app dispatches through this — no language-specific code outside the backend classes.

Features

Analysis — Finding Loop

8 presets — pick one based on what you want to find:

find-bugs find-dead-code find-business-rules summarize
find-bugs-sql find-business-rules-sql cleanup-stored-proc efficiency-review

The AI runs up to **3 iterations**. Each iteration it can request additional code (a method body, a class, a caller list) and the server fetches it via Roslyn before the next round.

Analysis — Finding Structure

Every finding the AI emits has these fields:

Field	What it is
<code>severity</code>	<code>bug</code> or <code>warning</code>
<code>confidence</code>	<code>high</code> (exact line + trigger named) or <code>low</code> (pattern real, full proof not visible)
<code>title</code>	One-sentence description
<code>description</code>	The failure path in one sentence
<code>filePath</code> + <code>lineNumber</code>	Where in the codebase
<code>codeSnippet</code>	The exact failing line

Low-confidence findings are **kept but dimmed** — never silently dropped.

Trace Mode

Builds a **call graph** starting from any method — callers, callees, or both.

- **Entry point:** type a method name, or click "Trace from here" in any file preview
- **Roslyn BFS:** server walks the call graph up to depth 5
- **Node types:** Normal (rectangle) · DbAccess (cylinder) · HttpCall (hexagon)
- **Mermaid:** diagram is generated by the server, not by the AI — always structurally correct
- **LLM synopses:** AI writes a 1–2 sentence summary for each node
- **Findings overlay:** after an analysis run, bug findings auto-decorate the matching nodes

Metrics Tab

Computes **code quality numbers** without using the AI — instant, deterministic.

Metric	C#	PL/SQL
Cyclomatic complexity	✓	✓
Nesting depth	✓	—
Method length	✓	—
Parameter count	✓	—
Empty-catch blocks	✓	—
Sync-over-async patterns	✓	—
Cursor declarations	—	✓
Swallowed WHEN OTHERS	—	✓

Results are **cached by file content hash** — reopening the tab on an unchanged workspace is instant.

Save to Repo

After any analysis or trace run, **Save to repo** writes a Markdown file into your repository at `docs/codeintel/`.

The file contains:

- All findings with severity, confidence, file/line, and code snippet
- A **Copilot Next Step** section — a ready-made prompt tailored to the preset
- An `INDEX.md` and `.codeintel-index.json` sidecar updated automatically

Paste `#file:docs/codeintel/your-report.md` into Copilot Chat to start triage.

Result Cache

Identical runs are **short-circuited** — no re-inference needed.

- Cache key = `preset + model name + SHA-256 of file contents`
- TTL = 7 days (configurable)
- Stored in SQLite
- A cache hit surfaces instantly with a status banner: **"Cache hit — reusing result from HH:MM (N findings)"**
- Editing any selected file invalidates the cache for that run automatically

Cancellation + Watchdogs

Every analysis and trace run has **three cancellation sources** linked together:

Source	Trigger
User cancel	Click the Cancel button
Idle watchdog	No new tokens for 90 seconds
Overall ceiling	Run exceeds 600 seconds total

Partial findings are always saved — a cancelled run can still be saved to repo.

Auto-Chunking

Files larger than the token budget (~5,000 tokens) are automatically **split into sequential chunks**.

- C#/TS/Java: splits at brace boundaries
- PL/SQL: splits at `END;` / `CREATE OR REPLACE`
- Each chunk runs one AI iteration with carry-over notes from previous chunks
- Status chip shows chunk progress during the run
- Capped at 8 chunks per file

Reliability Features

Feature	What it does
Rate limiting	5 runs/min per IP — rejects with <code>429</code> if exceeded
Secret scrubbing	Strips AWS keys, GitHub PATs, JWTs, PEM blocks before writing MD
Ignored findings	SHA-256 signature stored per workspace — suppresses on future runs
Findings diff	<code>GET /api/analysis/{id}/diff/{prevId}</code> → added / resolved / persisted
Workspace LRU cap	Max 3 loaded <code>.sln</code> workspaces in memory at once
<code>/healthz</code>	Liveness probe — always 200
<code>/readyz</code>	Readiness probe — checks LLM loaded + SQLite accessible

Done

Run it `dotnet run --project src\CodeIntel.Server`

Model loads on startup (~30s). Status dot turns green when ready.

Local LLM · Roslyn · ANTLR · SignalR · React 19 · SQLite